

Chapter 6: Embedded OS and graphics

6.1 Introduction

6.2 CMSIS

6.3 FreeRTOS and CMSIS-OS2

6.4 Embedded graphics with TouchGFX

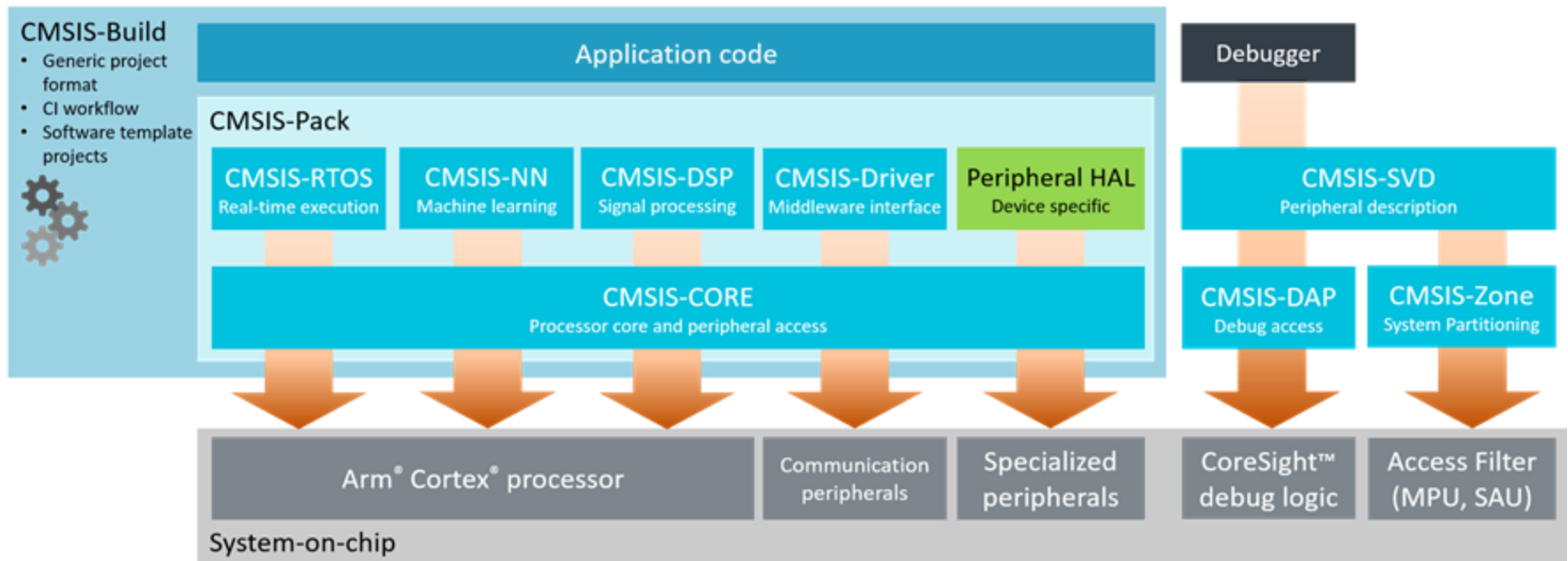
Introduction

- ❑ The embedded world is much much larger than STM32F4 + STM32CubeIDE!!!
- ❑ The larger view:
 - | Unlimited number of real-world application.
 - | MCU from different manufacturers: TI, NXP,...
 - | IDE from different vendors: IAR, Keil MDK,...
 - | Components from different providers.
 - | Product from different companies.
- ❑ More systematic and universal approaches are required!

ARM CMSIS v5

- ❑ STM32F4 = ARM Cortex-M4 CPU core + ARM core peripherals + vendor peripherals
 - | ARM Cortex-M4: CPU core designed by ARM, with standardized instruction set architecture from ARM → supported by many compilers.
 - | ARM core peripherals: peripherals that are part of ARM CPU core (NVIC, SysTick, FPU, debug...), accessed via ARM drivers.
 - | Peripherals: designed by vendors, most are standardized.
 - GPIO, I2C, SPI, timer, external interrupt, ADC/DAC,...
 - Accessed via vendor's driver or ARM driver interface.
- ❑ HAL: hardware abstraction layer provided by vendor
 - | Compatible between all vendor's MCU.
- ❑ CMSIS: “HAL” from ARM for ARM Cortex
 - | Compatible between all ARM Cortex MCU.
 - | ARM only provides prototype. Implementation is by vendor.

Development with ARM Cortex



https://arm-software.github.io/CMSIS_5/General/html/index.html

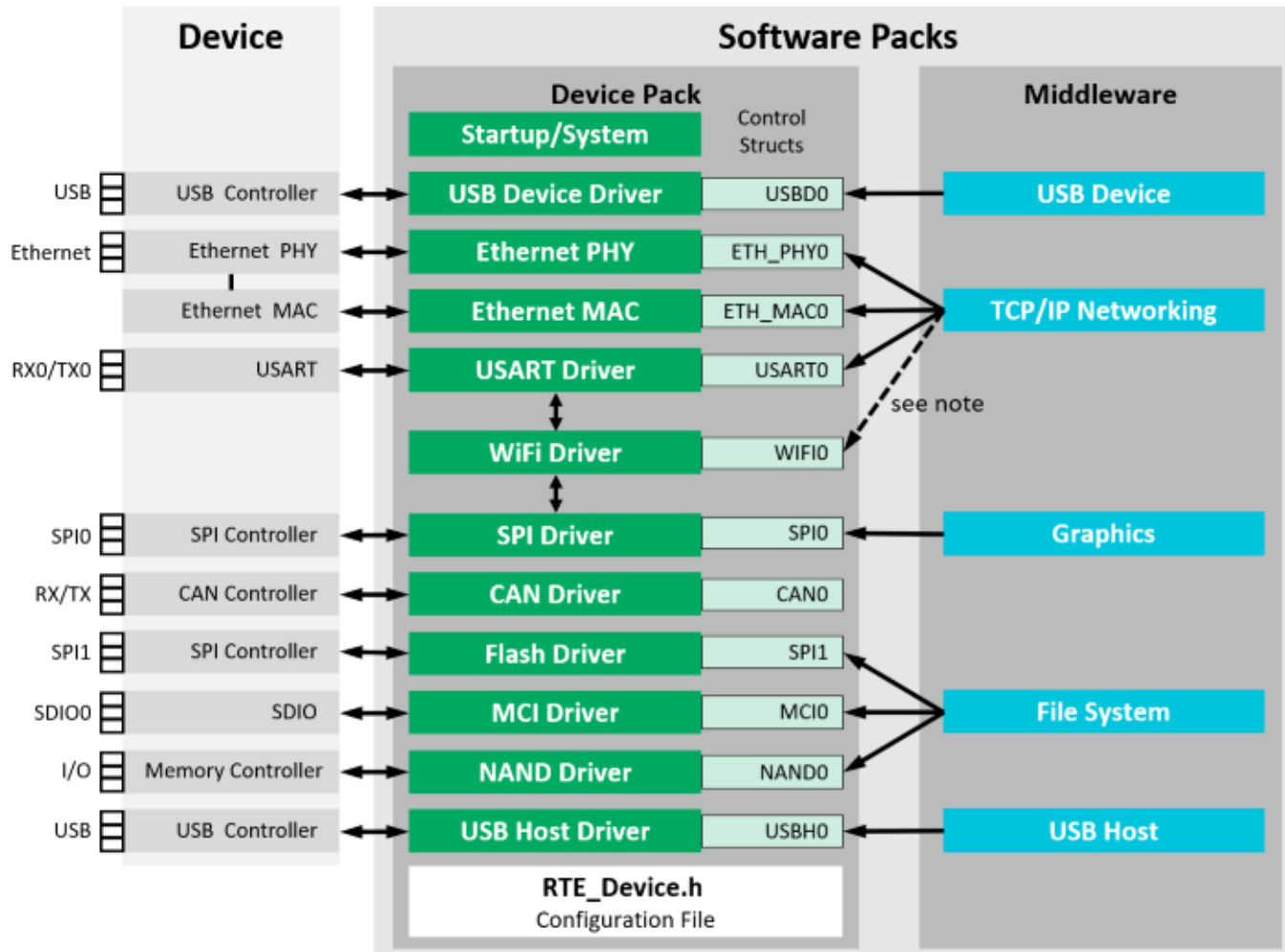
- ❑ CMSIS: vendor independent components provided by ARM.
- ❑ Peripheral HAL: provided by CPU vendor.
- ❑ Middleware: from 3rd party software vendors.

Development with ARM Cortex

- ❑ CMSIS Core: basic runtime provided by ARM, for accessing CPU and ARM's peripherals..
- ❑ CMSIS driver: generic driver for common peripherals (SPI, I2C, CAN, Ethernet,...), implemented by vendors.
- ❑ Extension libraries: APIs for extended functionalities optimized for ARM Cortex, implemented by ARM
 - | CMSIS-DSP: signal processing lib.
 - | CMSIS-NN: neural network lib.
 - | ...
- ❑ CMSIS-RTOS: real time OS interface, to use popular OS such as Free RTOS.

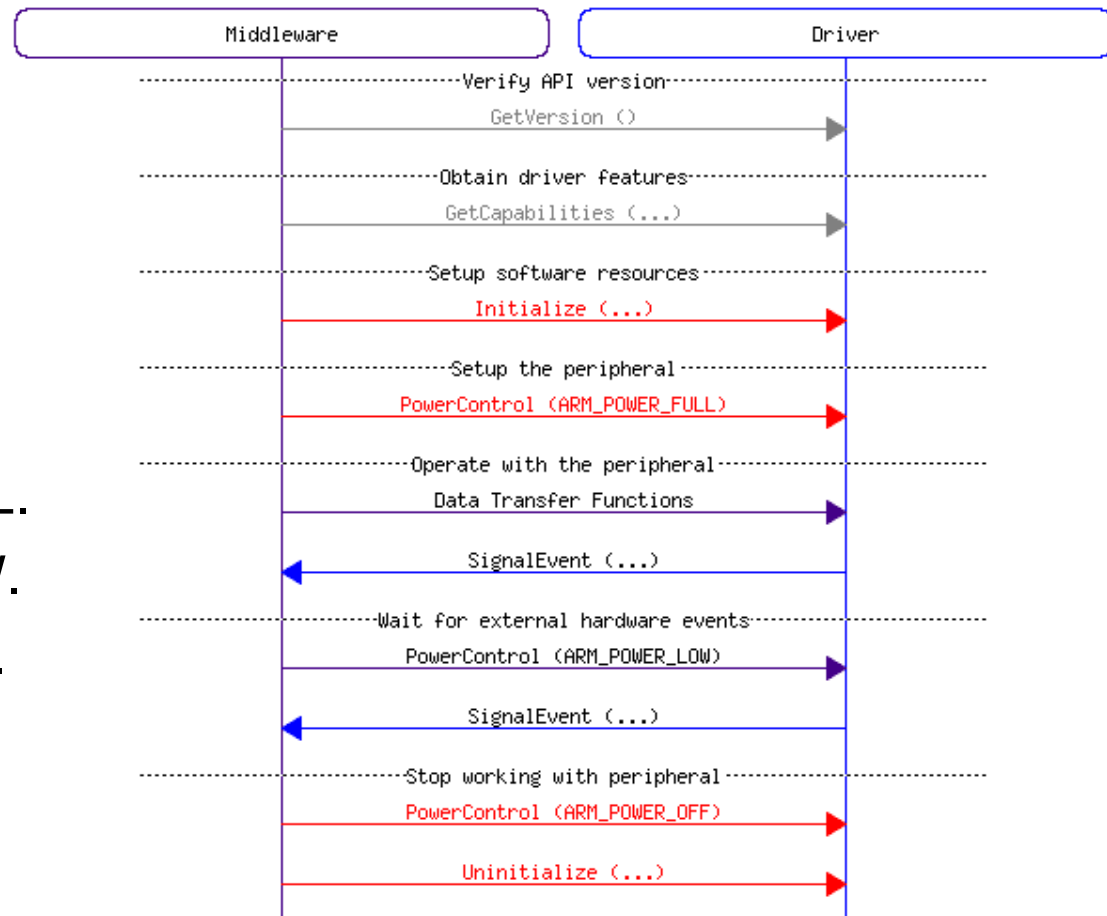
CMSIS driver

- ❑ Can replace HAL in some cases



Default APIs of CMSIS drivers

- ❑ `GetVersion()`.
- ❑ `GetCapabilities()`.
- ❑ `Initialize()`.
- ❑ `SignalEvent()`.
- ❑ `PowerControl()`
 - | `ARM_POWER_FULL`.
 - | `ARM_POWER_LOW`.
 - | `ARM_POWER_OFF`.
- ❑ `Uninitialize()`.
- ❑ `Control()`.



Example: CMSIS's SPI driver

```
typedef struct _ARM_DRIVER_SPI {  
    ARM_DRIVER_VERSION (*GetVersion) (void);  
    ARM_SPI_CAPABILITIES (*GetCapabilities) (void);  
    int32_t (*Initialize) (ARM_SPI_SignalEvent_t cb_event);  
    int32_t (*Uninitialize) (void);  
    int32_t (*PowerControl) (ARM_POWER_STATE state);  
    int32_t (*Send) (const void *data, uint32_t num);  
    int32_t (*Receive) (void *data, uint32_t num);  
    int32_t (*Transfer) (const void *data_out,  
                        void *data_in, uint32_t num);  
    uint32_t (*GetDataCount) (void);  
    int32_t (*Control) (uint32_t control, uint32_t arg);  
    ARM_SPI_STATUS (*GetStatus) (void);  
} const ARM_DRIVER_SPI;
```

ARM_DRIVER_SPI Driver_SPI1; // access functions for SPI1 interface

Example: CMSIS DSP lib

- ❑ Basic math functions
- ❑ Fast math functions
- ❑ Complex math functions
- ❑ Filtering functions
- ❑ Matrix functions
- ❑ Transform functions
- ❑ Motor control functions
- ❑ Statistical functions
- ❑ Support functions
- ❑ Interpolation functions
- ❑ Support Vector Machine functions (SVM)
- ❑ Bayes classifier functions
- ❑ Distance functions
- ❑ Quaternion functions

Development with ARM Cortex

- ❑ Advantages of ARM's CMSIS driver and ST's HAL:
 - | Standardized interface with hardware/peripherals via common APIs.
 - | Good compatibility between difference MCUs/line of products.
 - ❑ Problem:
 - | Single thread (main loop).
 - | Use main loop + interrupt + DMA to optimize CPU usage.
 - | Still not able to take advantage of CPU resource.
- ➔ Need an OS for multi-thread.

FreeRTOS

- ❑ Real-time OS for small embedded systems.
- ❑ Open source project, developed by Richard Barry (2003).
- ❑ Now maintain by Amazon (since 2017).
- ❑ Supports a lot of MCU series:
 - | 40+ architectures, 15+ toolchains.
 - | [Supported Devices - FreeRTOS™](#)

❑ Characteristics :

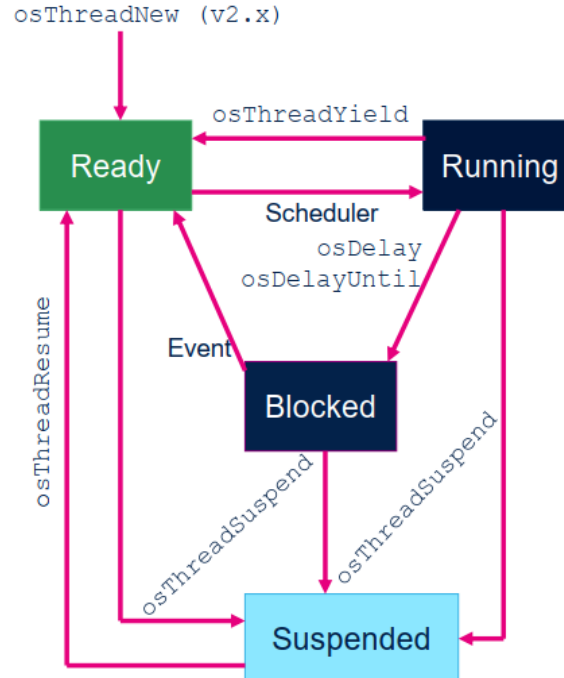
- | Small and reliable (passed formal verification).
 - FreeRTOS on STM32F4: ~10 KB flash ROM
- | Large developer community.
- | Support both free and commercial versions.

❑ Support all basic OS functionalities

- | Task management: create task, priority-based task scheduling.
- | Memory management.
- | Synchronization, inter-process communication with semaphore, queue,...

FreeRTOS task

- ❑ Task: an infinite execution thread (with an infinite loop).
- ❑ Runs with its own stack memory and priority.
- ❑ Can be created and deleted dynamically.
- ❑ Life-cycle: 4 states



Main APIs of FreeRTOS

❑ Create task

```
BaseType_t xTaskCreate(    TaskFunction_t pvTaskCode,  
                           const char * const pcName,  
                           configSTACK_DEPTH_TYPE usStackDepth,  
                           void *pvParameters,  
                           UBaseType_t uxPriority,  
                           TaskHandle_t *pxCreatedTask  
                           );
```

- | *pvTaskCode* : task routine
- | *pcName* : task name
- | *usStackDepth* : stack size (in words)
- | *pvParameters* : task param
- | *uxPriority* : priority of the task
- | *pxCreatedTask* : handle of the created task

Ex: create task

```
/* Task to be created. */
void vTaskCode( void * pvParameters )
{
    /* The parameter value is expected to be 1 as 1 is passed in the
    pvParameters value in the call to xTaskCreate() below.
    configASSERT( ( ( uint32_t ) pvParameters ) == 1 );

    for( ;; )
    {
        /* Task code goes here. */
    }
}

/* Function that creates a task. */
void vOtherFunction( void )
{
    BaseType_t xReturned;
    TaskHandle_t xHandle = NULL;

    /* Create the task, storing the handle. */
    xReturned = xTaskCreate(
        vTaskCode,          /* Function that implements the task. */
        "NAME",             /* Text name for the task. */
        STACK_SIZE,        /* Stack size in words, not bytes. */
        ( void * ) 1,       /* Parameter passed into the task. */
        tskIDLE_PRIORITY,  /* Priority at which the task is created. */
        &xHandle );         /* Used to pass out the created task's handle. */

    if( xReturned == pdPASS )
    {
        /* The task was created. Use the task's handle to delete the task. */
        vTaskDelete( xHandle );
    }
}
```

Main APIs of FreeRTOS

- ❑ Delete task

```
void vTaskDelete( TaskHandle_t xTask );
```

- ❑ Delete the task even when it is running

Main APIs of FreeRTOS

❑ Task control functions

- | `vTaskDelay`
- | `vTaskDelayUntil`
- | `uxTaskPriorityGet`
- | `vTaskPrioritySet`
- | `vTaskSuspend`
- | `vTaskResume`
- | `xTaskResumeFromISR`
- | `xTaskAbortDelay`

Main APIs of FreeRTOS

❑ System functions

- | taskYIELD
- | taskENTER_CRITICAL
- | taskEXIT_CRITICAL
- | taskENTER_CRITICAL_FROM_ISR
- | taskEXIT_CRITICAL_FROM_ISR
- | taskDISABLE_INTERRUPTS
- | taskENABLE_INTERRUPTS
- | vTaskStartScheduler
- | vTaskEndScheduler
- | vTaskSuspendAll
- | xTaskResumeAll
- | vTaskStepTick

CMSIS and FreeRTOS

- ❑ CMSIS provides a wrapper for embedded OS: CMSIS-OS.
- ❑ User software/application runs on CMSIS-OS.
- ❑ CMSIS-OS APIs are mapped to FreeRTOS API.

CMSIS_OS API v1.x	CMSIS_OS API v2.x	FreeRTOS API
osThreadCreate()	osThreadNew()	xTaskCreate()
osThreadGetId()	osThreadGetId()	xTaskGetCurrentTaskHandle()
osThreadTerminate()	osThreadTerminate() osThreadExit()	vTaskDelete()
osThreadYield()	osThreadYield()	taskYIELD()
osThreadSetPriority()	osThreadSetPriority()	vTaskPrioritySet()
osThreadGetPriority()	osThreadGetPriority()	uxTaskPriorityGet() uxTaskPriorityGetFromISR()
osDelay()	osDelay()	vTaskDelay()

CMSIS and FreeRTOS

❑ Examples:

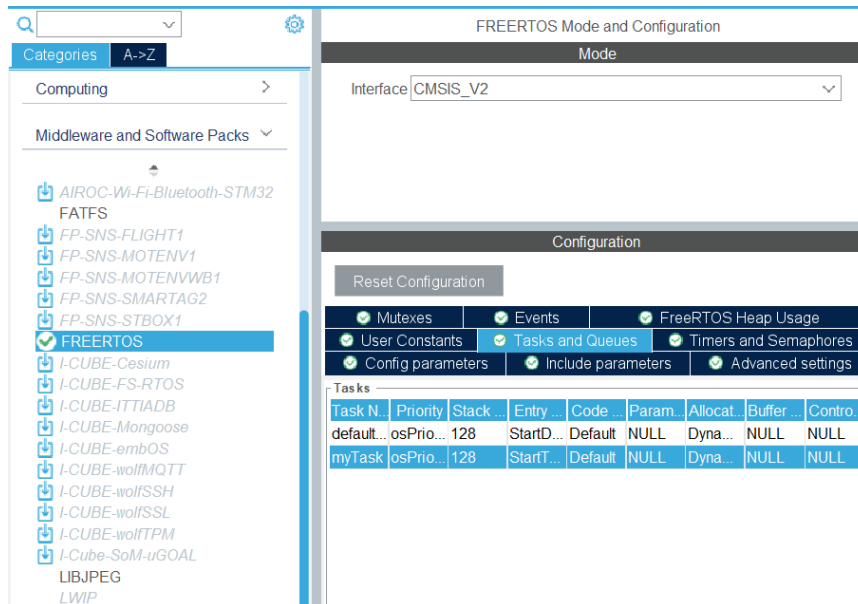
- | `osThreadId_t Task1Handle;`
- | `osThreadId_t osThreadNew(osThreadFunc_t func, void *argument, const osThreadAttr_t* attr)`
- | `osStatus_t osThreadTerminate(osThreadId_t thread_id)`
- | `osStatus_t osThreadYield(void)`
- | `osStatus_t osThreadResume(osThreadId_t thread_id)`
- | `osThreadState_t osThreadGetState(osThreadId_t thread_id)`
- | `osStatus_t osThreadSuspend(osThreadId_t thread_id)`

Exercise 1

- ❑ Build an application with 2 task
 - | Task 1: blink LED3 at 1 Hz
 - | Task 2: send data to PC via ST-LINK virtual COM port

Exercise 1

- ❑ Create a new project for STM32F429ZIT6
 - | PG13 and PG14 as GPIO Output.
 - | RCC and System clock to set CPU HCLK at 90 MHz. (Note: Input frequency = 8 MHz in Clock configuration).
 - | Enable USART1 as Asynchronous 115200 bps.
 - | Set Timebase source in SYS: TIM6 (because SysTick is used for FreeRTOS).
- ❑ Configure FreeRTOS in the Middleware tab



Enable the CMSIS_V2 interface

Declare an additional task:
myTask, with the same
priority with the default task.

Exercise 1

❑ Add code for the two task routines

```
289 /* USER CODE BEGIN Header_StartDefaultTask */
290 /**
291  * @brief Function implementing the defaultTask thread.
292  * @param argument: Not used
293  * @retval None
294  */
295 /* USER CODE END Header_StartDefaultTask */
296 void StartDefaultTask(void *argument)
297 {
298     /* USER CODE BEGIN 5 */
299     /* Infinite loop */
300     for(;;)
301     {
302         osDelay(500);
303         HAL_GPIO_TogglePin(GPIOD, GPIO_PIN_14);
304     }
305     /* USER CODE END 5 */
306 }
307
308 /* USER CODE BEGIN Header_StartTask02 */
309 /**
310  * @brief Function implementing the myTask thread.
311  * @param argument: Not used
312  * @retval None
313  */
314 /* USER CODE END Header_StartTask02 */
315 void StartTask02(void *argument)
316 {
317     /* USER CODE BEGIN StartTask02 */
318     /* Infinite loop */
319     for(;;)
320     {
321         HAL_GPIO_TogglePin(GPIOD, GPIO_PIN_13);
322         HAL_UART_Transmit(&huart1, "Hello from user task\n", 21, 10);
323
324         osDelay(500);
325     }
326     /* USER CODE END StartTask02 */
327 }
```

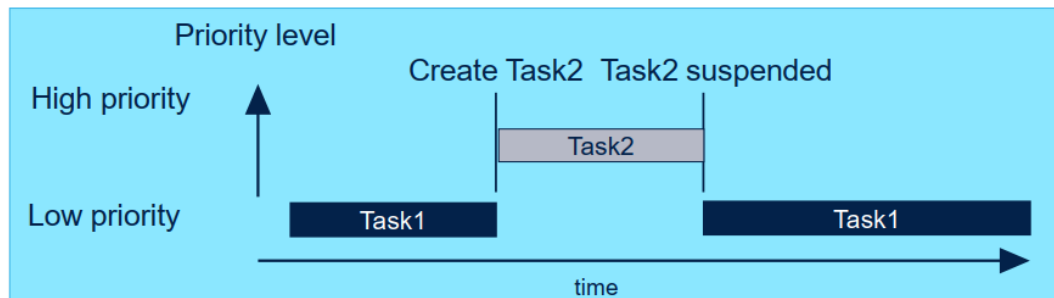
Exercise 1

- ❑ Run and observe:
 - | LED blinks at 1 Hz.
 - | The text “Hello from user task” sent to PC and displays on Hercules.
- ❑ Note: the 2 tasks runs just like we have two main loops running in parallel.
 - ➔ CPU resource can by used more efficiently.

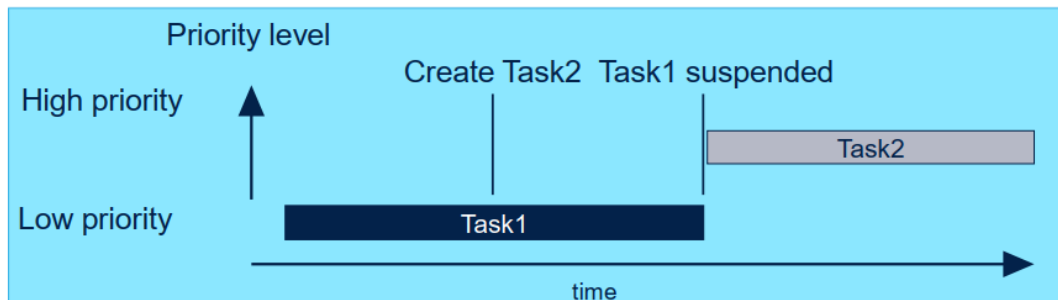
Task scheduling and context switching

❑ Scheduling: decide task execution order

- | Preemptive: task with higher priority can preempt the task with lower priority, tasks with the same priority follow round robin (review Operating Systems course).
- | Non preemptive: task that arrives earlier will execute first.



Pre-emptive scheduling

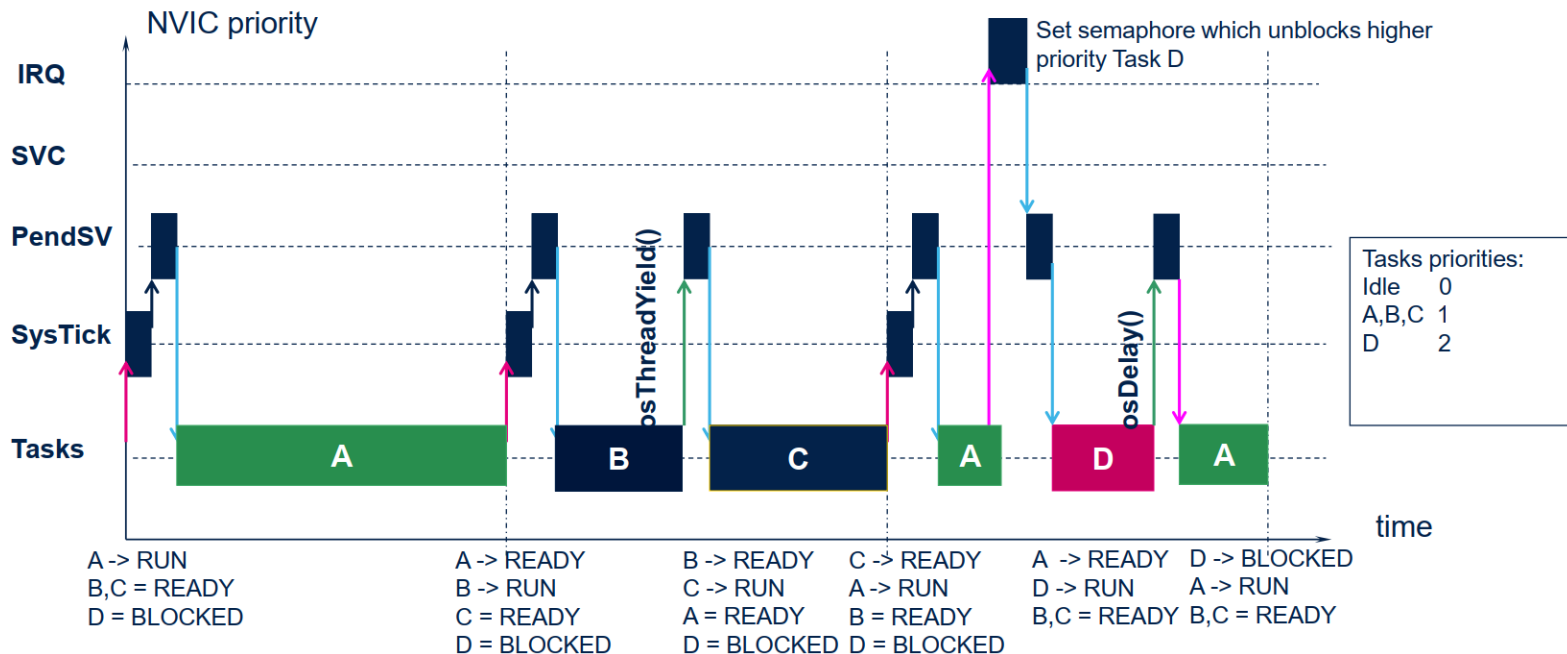


Non Pre-emptive scheduling

Task scheduling and context switching

Context switching

- | CPU pauses 1 task's execution, change to another task.
- | Occur when: SysTick, osDelay() call, thread yield, interrupt, semaphore block,...
 - Note: SysTick
 - Note: HAL_Delay() is a locking function, run with the task's priority.



Exercise 2: Task scheduling and priority

- ❑ Create a new project (or reuse project from Ex.1)
 - | Set RCC + CPU clock at 90 MHz.
 - | Set Timebase source in SYS as TIM6.
 - | Enable USART1, and set PG13, PG14 as GPIO Output.
 - | Enable FreeRTOS in Middleware and Software pack.
 - Create 2 task myTask1 and myTask2.
 - Note: set myTask1 with osPriorityNormal1 (higher than myTask2).

The 'Edit Task' dialog box for myTask1 shows the following configuration:

Task Name	myTask1
Priority	osPriorityNormal1
Stack Size (Words)	128
Entry Function	StartTask1
Code Generation Option	Default
Parameter	NULL
Allocation	Dynamic
Buffer Name	NULL
Control Block Name	NULL

Buttons: OK, Cancel

The 'Edit Task' dialog box for myTask2 shows the following configuration:

Task Name	myTask2
Priority	osPriorityNormal
Stack Size (Words)	128
Entry Function	StartTask2
Code Generation Option	Default
Parameter	NULL
Allocation	Dynamic
Buffer Name	NULL
Control Block Name	NULL

Buttons: OK, Cancel

Thread control block

Exercise 2: Task scheduling and priority

❑ Generate code and add the below code

```
71  /* USER CODE BEGIN PFP */
72  void SendChar(char c)
73  {
74      uint8_t str[] = {c, '\r', '\n'};
75      HAL_UART_Transmit(&huart1, str, 3, 10);
76  }
77  /* USER CODE END PFP */
```

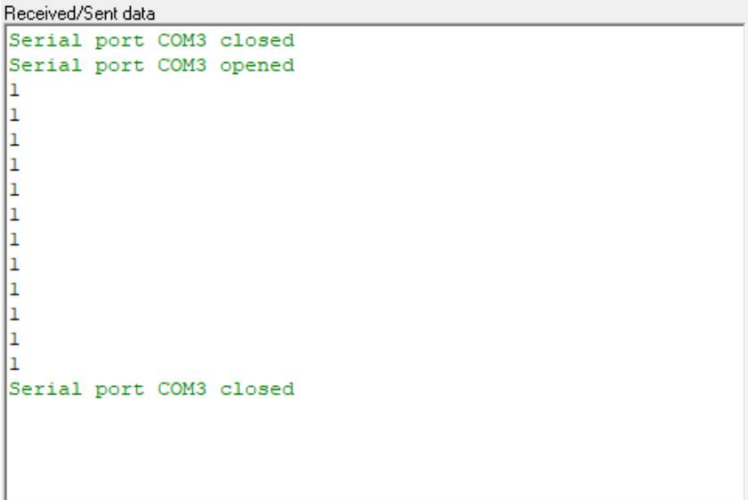
```
294 void StartTask1(void *argument)
295 {
296     /* USER CODE BEGIN 5 */
297     /* Infinite loop */
298     for(;;)
299     {
300         SendChar('1');
301         HAL_Delay(200);
302         //osDelay(200);
303     }
304     /* USER CODE END 5 */
305 }
```

```
314 void StartTask2(void *argument)
315 {
316     /* USER CODE BEGIN StartTask2 */
317     /* Infinite loop */
318     for(;;)
319     {
320         SendChar('2');
321         osDelay(200);
322     }
323     /* USER CODE END StartTask2 */
324 }
```

Note: Task1 routine uses HAL_Delay()

Exercise 2: Task scheduling and priority

- ❑ Build and run code on STM32 board. Observe the content on PC's Hercules window.
 - | Hercules only shows "1" characters → **Task 2 does not run!!!**
 - | Reason: Task 1 uses HAL_Delay and has higher priority, it consumes 100% of CPU time.
- ❑ How to allow Task 2 to run:
 - | Keep priority of Task 1 and use osDelay() instead of HAL_Delay().
 - | Or, keep Task 1 code and reduce its priority to be smaller or equal than Task 2's priority.



```
Received/Sent data
Serial port COM3 closed
Serial port COM3 opened
1
1
1
1
1
1
1
1
1
1
1
1
1
1
Serial port COM3 closed
```

➔ *Important: task loop must always has osDelay()*

Example 3: Create and delete task

- ❑ Write code so that the tasks work as follow:
 - | Task 1: send characters 'a', 'b',... to PC via UART with the rate of one character per second.
 - | Task 2: detect the event button pressed on PA0. If the button is pressed, Task 1 changes to sending 'A', 'B',... Instead of 'a', 'b'...
 - | Note: changes must be immediately effective without having to wait for the duration of 1000ms in Task 1.

Example 3: Create and delete task

- ❑ Reuse the project from Ex.2
- ❑ Config PA0 as GPIO_Input.
- ❑ Declare the variable task1Count
- ❑ Task 1: receive param when initializing.
- ❑ Task 2: detect button press, delete and recreate Task 1.

```
71 /* USER CODE BEGIN PFP */
72 void SendChar(char c)
73 {
74     uint8_t str[] = {c, '\r', '\n'};
75     HAL_UART_Transmit(&huart1, str, 3, 10);
76 }
77 /* USER CODE END PFP */
78
79 /* Private user code -----
80 /* USER CODE BEGIN 0 */
81 uint8_t task1Count = 'a';
82 /* USER CODE END 0 */
```

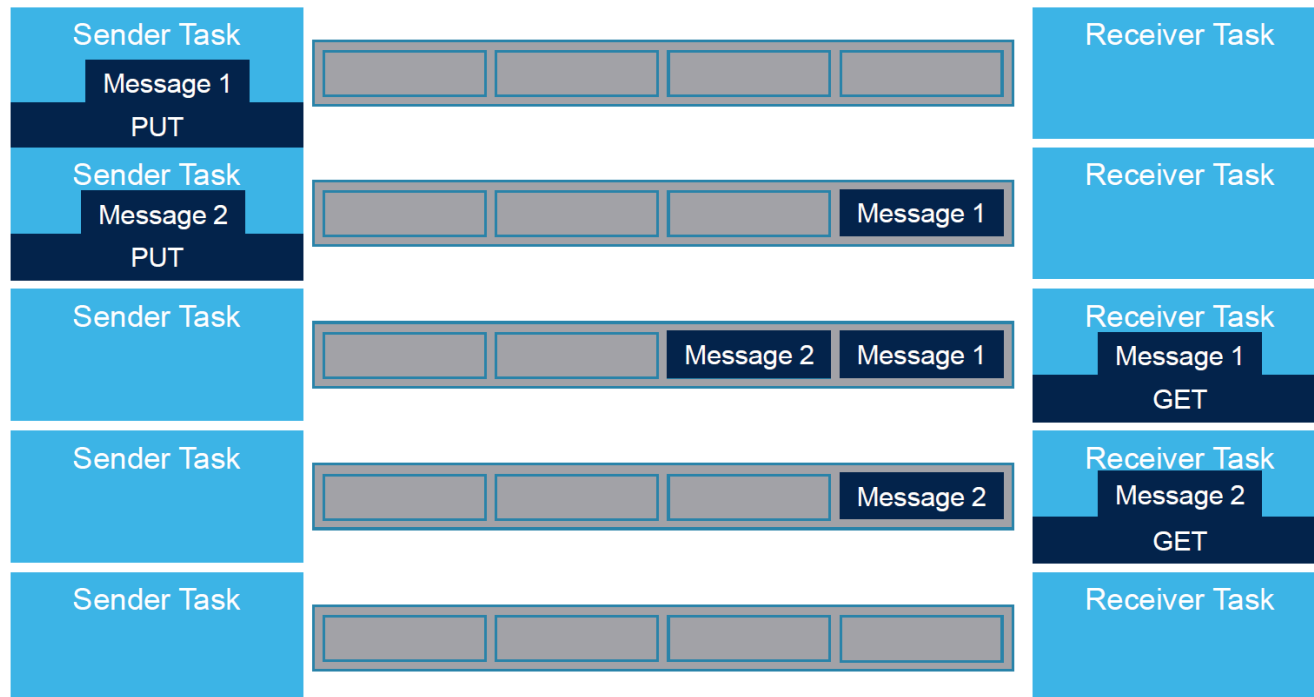
```
288 /* USER CODE END Header_StartTask01 */
289 void StartTask01(void *argument)
290 {
291     /* USER CODE BEGIN 5 */
292     if (argument == NULL)
293         task1Count = 'a';
294     else
295     {
296         uint8_t *p = (uint8_t *)argument;
297         task1Count = *p;
298     }
299     /* Infinite loop */
300     for(;;)
301     {
302         SendChar(task1Count);
303         task1Count++;
304         osDelay(1000);
305     }
306     /* USER CODE END 5 */
307 }
315 /* USER CODE END Header_StartTask02 */
316 void StartTask02(void *argument)
317 {
318     /* USER CODE BEGIN StartTask02 */
319     /* Infinite loop */
320     for(;;)
321     {
322         if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0)
323             == GPIO_PIN_SET)
324         {
325             uint8_t c = 'A';
326             osThreadTerminate(myTask01Handle);
327             myTask01Handle = osThreadNew(StartTask01, &c,
328                 &myTask01_attributes);
329             osDelay(500);
330         }
331         osDelay(1);
332     }
333     /* USER CODE END StartTask02 */
334 }
```

RTOS kernel primitives

- ❑ Queues
- ❑ Queue Sets
- ❑ Stream Buffers
- ❑ Message Buffers
- ❑ Semaphore / Mutexes

Task communication

- ❑ IPC: inter-process communication, for sending data between tasks.
 - | Queue, Queue sets, Buffer, Semaphore...
- ❑ Queue: sending integers/pointers between tasks



Main queue APIs

- ❑ `const osMessageQueueAttr_t attr= { .name= "Queue1" };`
- ❑ `osMessageQueueId_t osMessageQueueNew(uint32_t msg_cnt,
uint32_t msg_size,
const osMessageQueueAttr_t *attr);`
- ❑ `osStatus_t osMessageQueueDelete(osMessageQueueId_t queue_id);`
- ❑ `osStatus_t osMessageQueuePut(osMessageQueueId_t queue_id,
const void *msg_ptr,
uint8_t msg_prio,
uint32_t timeout);`
- ❑ `osStatus_t osMessageQueueGet(osMessageQueueId_t queue_id,
const void *msg_ptr,
uint8_t *msg_prio,
uint32_t timeout);`
- ❑ `uint32_t osMessageQueueGetCount(osMessageQueueId_t queue_id);`

Exercise 4

- ❑ Reuse project from previous exercise: 2 tasks
 - | Task 1: blinks LED3 at 1 Hz
 - | Task 2: send data to PC via ST-LINK virtual COM port
 - ❑ Extend new function:
 - | Create a new Task 3 for polling status of the button on PA0.
 - | If button is pressed, data is sent continuously to PC.
 - | Data sending to PC is done by Task 2.
 - ❑ Communication between Task 2 and Task 3 is required
- using message queue for task communication

Exercise 4

- ❑ Create new task (Task 3).
- ❑ Create message queue for data transfers between tasks

The screenshot shows the FreeRTOS configuration tool interface. At the top, there are several tabs: 'Mutexes', 'Events', 'FreeRTOS Heap Usage', 'User Constants', 'Tasks and Queues' (which is selected), 'Timers and Semaphores', 'Config parameters', 'Include parameters', and 'Advanced settings'. Below these tabs, there is a 'Tasks' section with an 'Edit Queue' dialog box open. The dialog box contains the following fields:

- Queue Name: myQueue01
- Queue Size: 16
- Item Size: uint16_t
- Allocation: Dynamic
- Buffer Name: NULL
- Buffer size: n/a
- Control Block Name: NULL

Below the dialog box, there is a 'Queues' table with the following data:

Queue Name	Queue Size	Item Size	Allocation	Buffer Name	Control Block Name
myQueue01	16	uint16_t	Dynamic	NULL	NULL

At the bottom right of the 'Queues' section, there are 'Add' and 'Delete' buttons.

Exercise 4

❑ Code for Task 2 and Task 3

```
void StartTask03(void *argument)
{
    /* USER CODE BEGIN StartTask03 */
    /* Infinite loop */
    for(;;)
    {
        if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == GPIO_PIN_SET)
        {
            uint8_t data = 'X';
            osMessageQueuePut(myQueue01Handle, &data, 0, 10);
            osDelay(200);
        }
        osDelay(1);
    }
    /* USER CODE END StartTask03 */
}

void StartTask02(void *argument)
{
    /* USER CODE BEGIN StartTask02 */
    /* Infinite loop */
    for(;;)
    {
        uint8_t res;
        if (osMessageQueueGetCount(myQueue01Handle) > 0)
        {
            osMessageQueueGet(myQueue01Handle, &res, NULL, osWaitForever);
            if (res == 'X')
                HAL_UART_Transmit(&huart1, "Button pressed\n", 15, 10);
        }

        osDelay(1);
    }
    /* USER CODE END StartTask02 */
}
```

Embedded graphics with TouchGFX

- ❑ Combine the CPU's hardware resource and the multithread capability of OS → more complicated system can be built, for example embedded graphics.
- ❑ TouchGFX là framework do ST cung cấp, cho phép xây dựng giao diện đồ họa trên các hệ thống nhúng dựa trên STM32.
- ❑ Tích hợp chặt chẽ với bộ công cụ của ST.
 - | Tạo project, cấu hình giao diện, sinh mã, chạy mô phỏng trên TouchGFX Designer.
 - | Phát triển phần mềm trên STM32CubeIDE.
- ❑ Phiên bản mới nhất: 4.25.0

Embedded graphics với TouchGFX

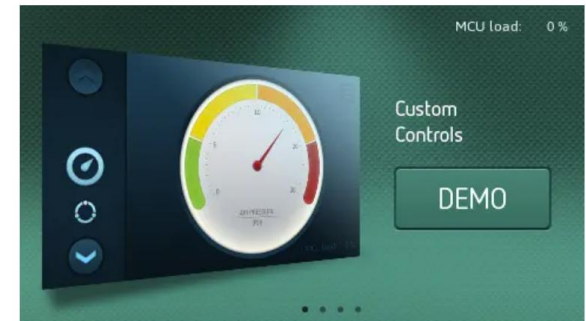
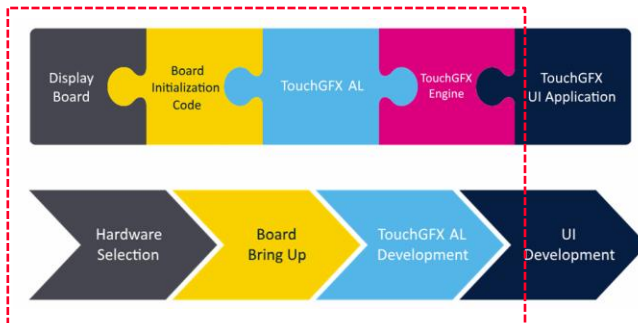
❑ Đặc điểm embedded graphics

- | Giao diện đồ họa trên các hệ nhúng ít tài nguyên, không có GPU.
- | Độ sâu màu thấp: grayscale, RGB 8 bit hoặc RGB 16 bit.
- | Độ phân giải màn hình nhỏ, kích thước RAM nhỏ.

❑ TouchGFX:

- | Pixel format tiêu chuẩn RGB565.
- | Single hoặc double frame buffer.
- | Sử dụng FreeRTOS để quản lý task.
- | Các bước phát triển ứng dụng.

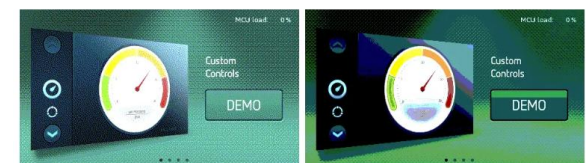
Được cấu hình
sẵn cho các
development
board



Original RGB888 image



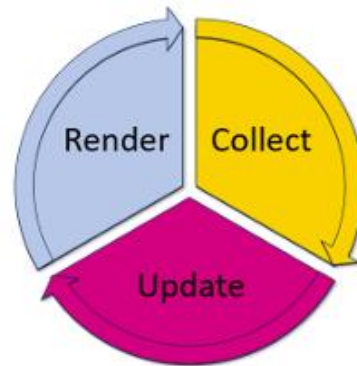
Converted RGB565 images with and without dithering



Chất lượng hình ảnh theo pixel format

Embedded graphics với TouchGFX

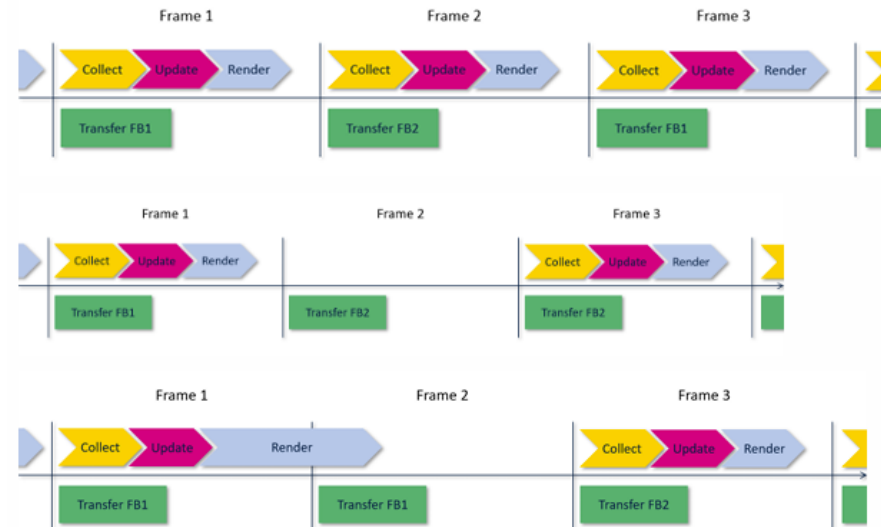
❑ Main loop



1. Collecting events
 - Touchscreen and buttons
 - Backend (Ticks)
2. Updating
 - Changes the “state” of the widgets based on the events collected
3. Rendering
 - TouchGFX draws relevant updates in the framebuffer

❑ Refresh rate

- | Theo thông số của màn hình (thường ~60 Hz).
- | Sự kiện tick() trên UI.
- | Tính toán chậm sẽ làm giảm framerate thực.



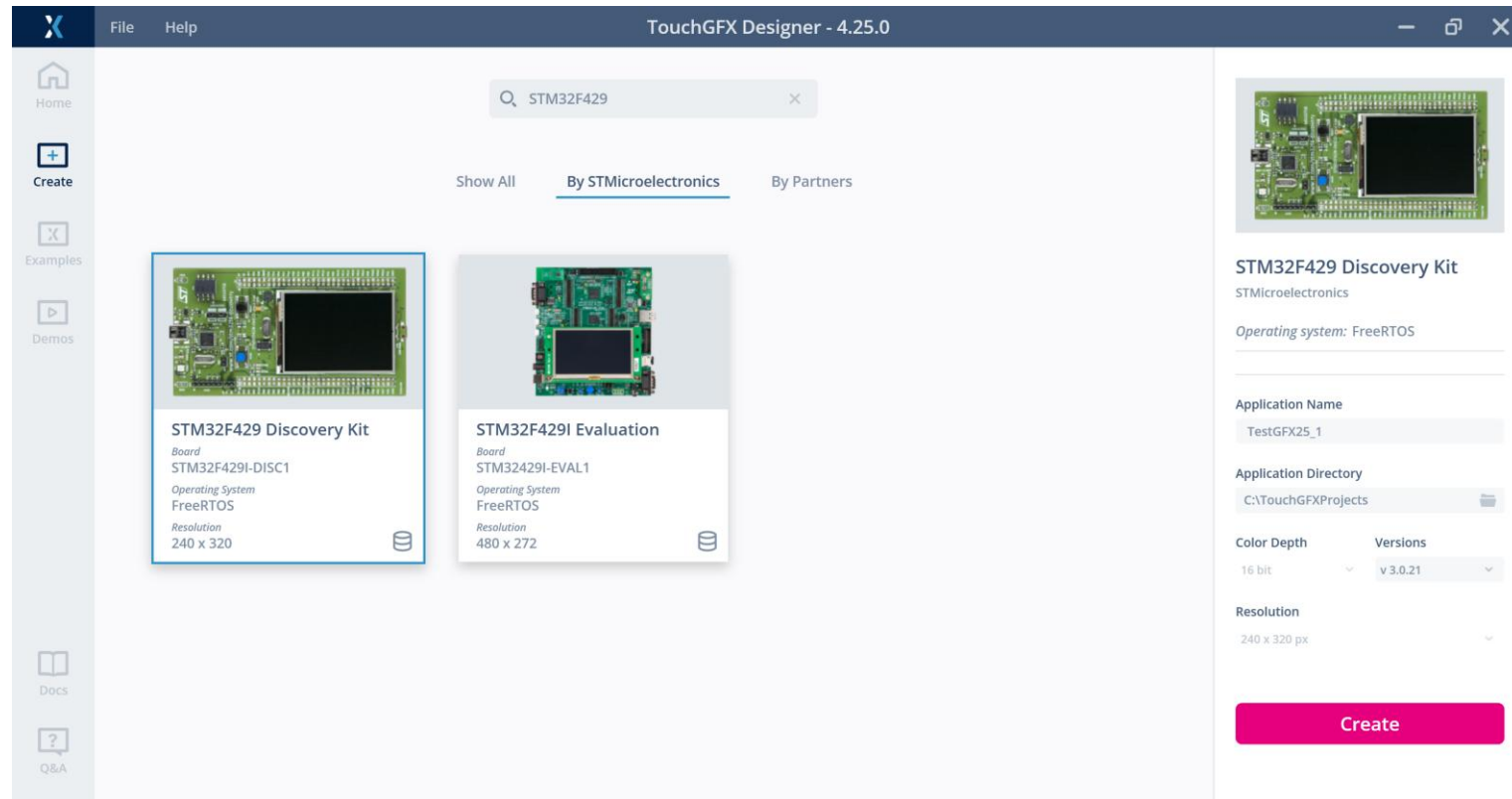
Embedded graphics với TouchGFX

❑ Các UI components/widgets: tham khảo [UI Components | TouchGFX Documentation](#)

Buttons ▼	Images ▼	Containers ▼	Miscellaneous ▼
Button	Image	Container	Slider
Button With Label	Scalable Image	Scrollable Container	Text Area
Button With Icon	Tiled Image	Swipe Container	Analog Clock
Toggle Button	Animated Image	List Layout	Digital Clock
Radio Button	Texture Mapper	Modal Window	Static Graph
Repeat Button	SVG Image	Scroll List	Dynamic Graph
Flex Button	QR Code	Scroll Wheel	Gauge
		Slide Menu	Video
Shapes ▼	Progress Indicators ▼		
Box	Box Progress		
Box With Border	Image Progress		
Line	Text Progress		
Circle	Line Progress		
Shape	Circle Progress		

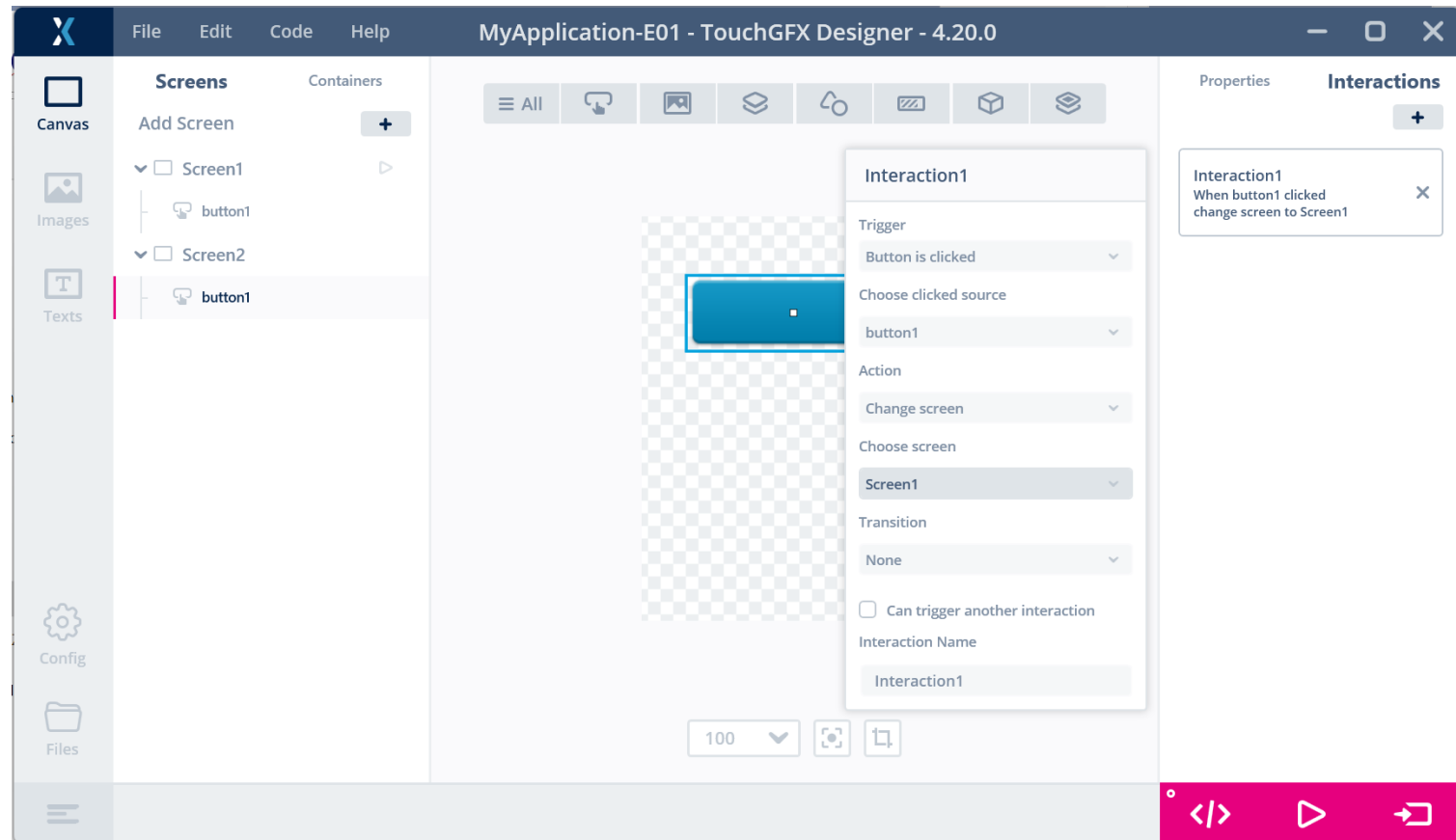
Ví dụ: Tạo project với TouchGFX Designer

❑ Tạo project cho STM32F429I-DISCO



Tạo project với TouchGFX Designer

- ❑ Tạo giao diện, interaction
- ❑ Generate code, chạy thử trên board

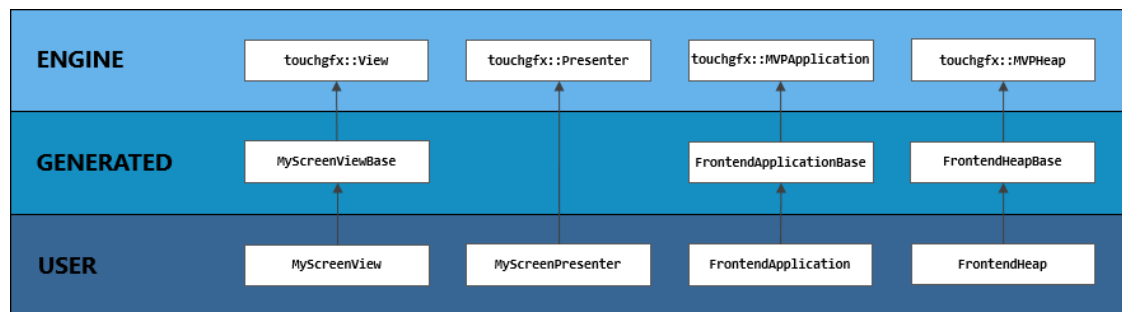
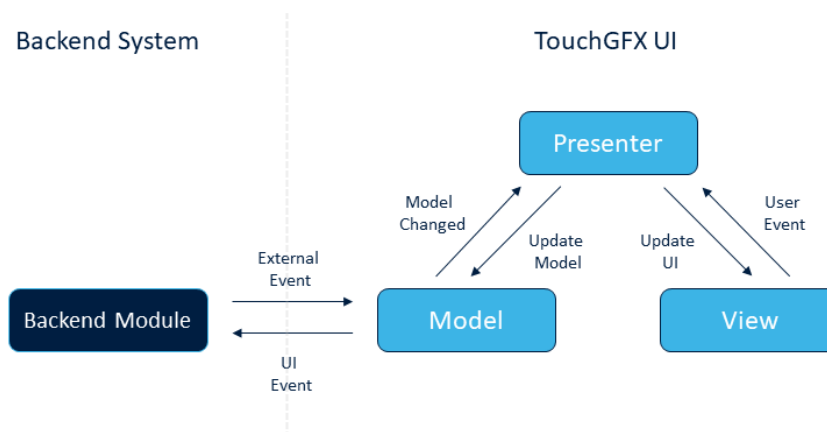


Cấu trúc project

- IDE STM32F429I-DISCO
 - > Binaries
 - > Includes
 - > Application
 - > Startup
 - > User
 - > generated
 - FrontendApplication.cpp
 - Model.cpp
 - Screen1Presenter.cpp
 - Screen1View.cpp
 - Screen2Presenter.cpp
 - Screen2View.cpp
 - TouchGFX
 - freertos.c
 - main.c
 - stm32f4xx_hal_msp.c
 - stm32f4xx_hal_timebase_tim.c
 - stm32f4xx_it.c
 - syscalls.c
 - systemem.c
 - > Debug
 - > Drivers
 - > Middlewares
 - STM32F429I-DISCO.launch
 - STM32F429ZITX_FLASH.Id
 - STM32F429ZITX_RAM.Id

Backend System

TouchGFX UI



Bài tập

- ❑ Hãy tìm hiểu các đoạn code khởi tạo trong hàm main

Animation với TouchGFX

❑ Timebased update (giống game loop)

❑ Vd:

```
void handleTickEvent()
{
    Screen1ViewBase::handleTickEvent();    // Call superclass eventhandler
    tickCounter += 1;
    if (tickCounter == 600)
    {
        myWidget.startFadeAnimation(0, 20); // Fade to 0 = invisible in 20 frames
    }
}
```

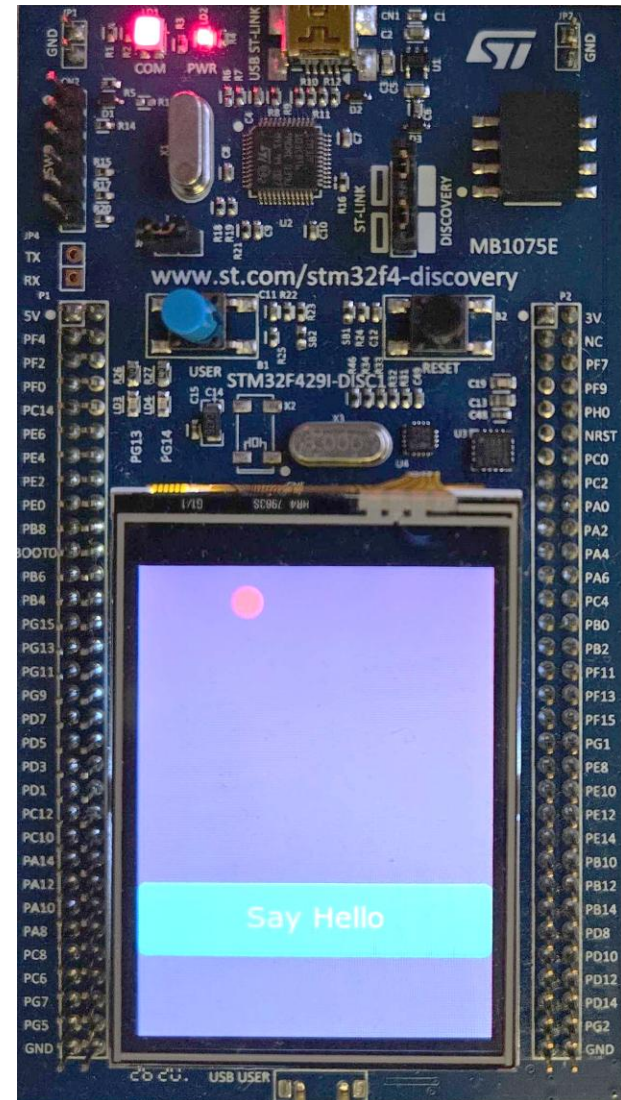
```
void handleTickEvent()
{
    Screen1ViewBase::handleTickEvent();    // Call superclass eventhandler
    tickCounter += 1;
    if (tickCounter == 10)
    {
        box1.setColor(Color::getColorFromRGB(0xFF, 0x00, 0x00)); // Set color to red
        box1.invalidate();                                         // Request redraw
    }
}
```

Backend communication

- ❑ Giao tiếp giữa các đối tượng TouchGFX với hardware.
- ❑ 2 phương án
 - | Polling trực tiếp từ hàm tick của model, hoặc tick handler của view.
 - | Tạo riêng 1 task giao tiếp với phần cứng, đẩy dữ liệu sang TouchGFX qua queue của FreeRTOS.

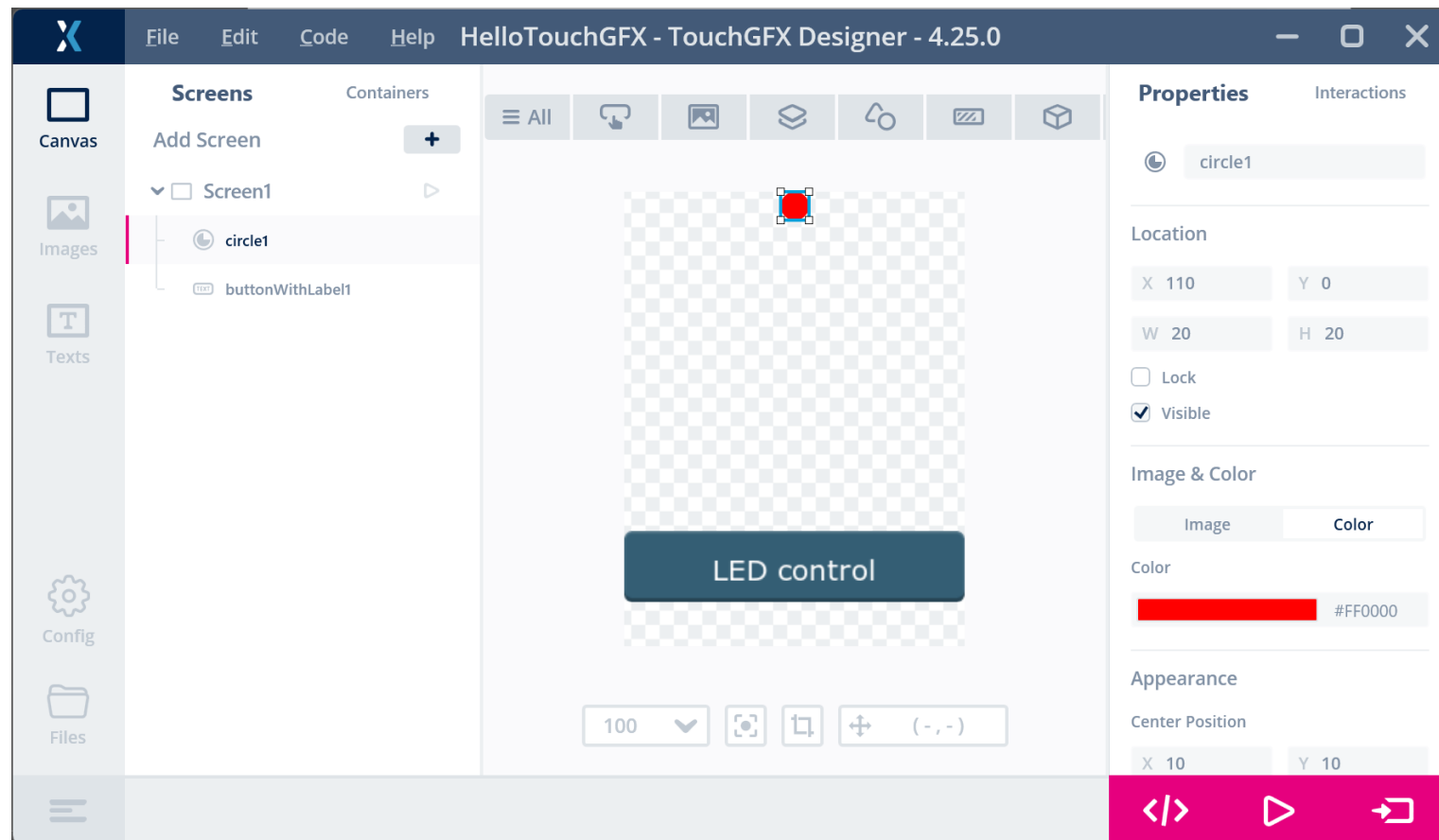
Bài tập

- ❑ Thiết kế giao diện với 2 widget như hình.
- ❑ Lập trình thực hiện
 - | Khi bấm nút mềm trên giao diện thì đèn LED bật/tắt.
 - | Khi bấm nút cứng PA0 (nút xanh) thì hình tròn đỏ di chuyển.



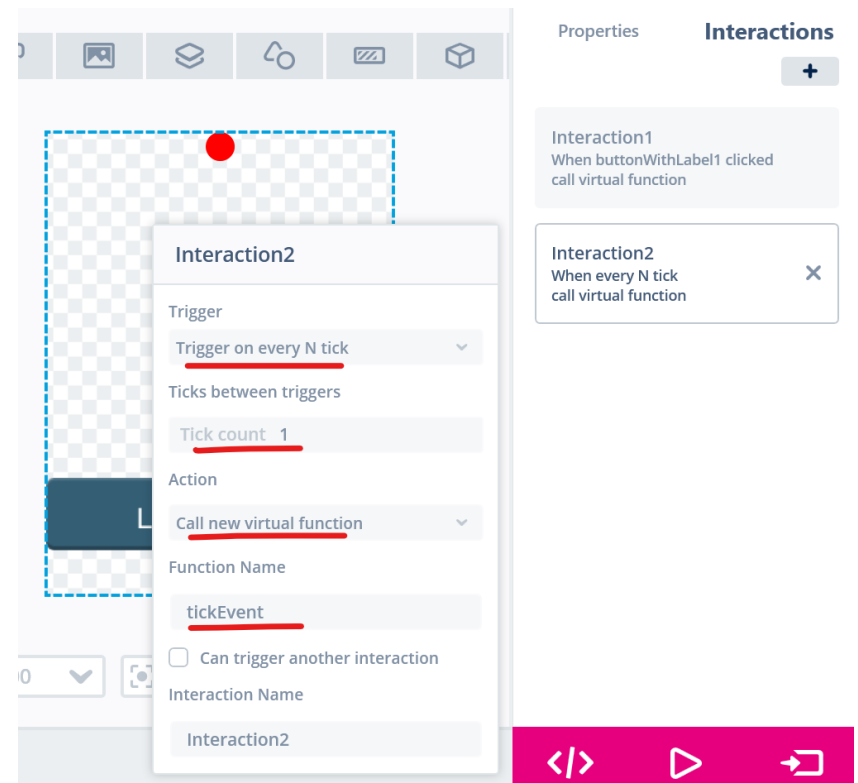
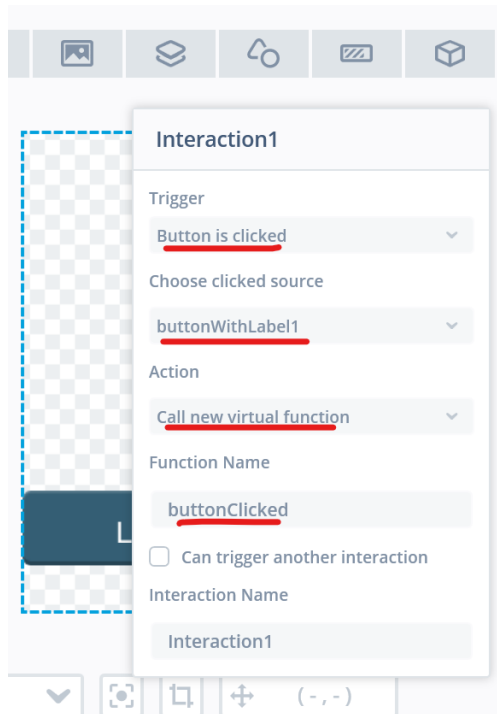
Bài tập

- ❑ Tạo project mới trong TouchGFX Designer (chọn STM32F429I-DISCO Rev E).
- ❑ Project có 1 screen với 2 widget buttonWithLabel1 và circle1.



Bài tập

- ❑ Tạo các Interactions (sự kiện).
- ❑ Sinh code (bấm F4).



Bài tập

- ❑ Cập nhật hàm `MX_GPIO_Init()` trong hàm `main` để khởi tạo chân PA0 là `GPIO_Input`, và PG13, PG14 là `GPIO_Output` (tham khảo code các tuần trước).

```
/* USER CODE BEGIN MX_GPIO_Init_2 */
/*Configure LED pins : PG13 PG14 */
GPIO_InitStruct.Pin = GPIO_PIN_13|GPIO_PIN_14;
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH;
HAL_GPIO_Init(GPIOD, &GPIO_InitStruct);

/*Configure GPIO pin : PA0 */
GPIO_InitStruct.Pin = GPIO_PIN_0;
GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
GPIO_InitStruct.Pull = GPIO_NOPULL;
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
/* USER CODE END MX_GPIO_Init_2 */
```

Bài tập

- ❑ Lập trình hàm của default task để bắt sự kiện bấm nút PA0 và gửi dữ liệu vào queue khi nút được bấm.

```
/* USER CODE END Header_StartDefaultTask */
void StartDefaultTask(void *argument)
{
    /* USER CODE BEGIN 5 */
    /* Infinite loop */
    for(;;)
    {
        if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == GPIO_PIN_SET)
        {
            uint8_t data = 'X';
            osMessageQueuePut(myQueue01Handle, &data, 0, 10);
        }

        osDelay(1);
    }
    /* USER CODE END 5 */
}
```

Bài tập

- ❑ Thêm biến tickCount vào khai báo lớp Screen1View

```
class Screen1View : public Screen1ViewBase
{
public:
    Screen1View();
    virtual ~Screen1View() {}
    virtual void setupScreen();
    virtual void tearDownScreen();

    void buttonClicked();
    void tickEvent();
protected:
    uint32_t tickCount = 0;
};
```

Bài tập

❑ Xử lý các hàm `buttonClicked()` và `tickEvent()` trong lớp `Screen1View`

```
void Screen1View::buttonClicked()
{
    HAL_GPIO_TogglePin(GPIOG, GPIO_PIN_13);
}

extern osMessageQueueId_t myQueue01Handle;
void Screen1View::tickEvent()
{
    tickCount += 2;
    tickCount = tickCount % 240;

    float x = tickCount / 55.0f;
    float y = sin(x)+sin(2*x)+sin(3*x)+1;

    uint8_t res;
    if (osMessageQueueGetCount(myQueue01Handle) > 0)
    {
        osMessageQueueGet(myQueue01Handle, &res, NULL, osWaitForever);
        if (res == 'X')
        {
            circle1.moveTo((int16_t)floor(x*55),200-(int16_t)floor(y*50));
            circle1.invalidate();
        }
    }
}
```

Bài tập

- ❑ Dịch và chạy trên kit, quan sát kết quả.
- ❑ Giải thích hoạt động của chương trình.